

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Student Name	K. Hansikha
Experiment No.	PBL Part 2 – Task 6
Title	Node.js Built-in Modules – os, path and fs

1. TITLE

Node.js Built-in Modules – os, path and fs

2. AIM

To understand Node.js modules and demonstrate the use of the os, path, and fs built-in modules for system information, file paths, and file handling.

3. OBJECTIVE

- To understand modularity in Node.js.
 - To use the os module for operating-system information.
 - To use the path module for file-path operations.
 - To use the fs module for creating and reading files.
 - To execute a Node.js program from the terminal.
-

4. THEORY

Node.js provides built-in modules that can be imported without installing external packages. The os module provides operating-system information such as platform and CPU details. The path module provides utilities for working with file and directory paths. The fs module provides file-system operations such as writing and reading files.

5. ALGORITHM / PROCEDURE

1. Create a Node.js file named app.js.
 2. Import the os, path, and fs modules using require().
 3. Read basic operating-system information using os.
 4. Create and display a file path using path.join().
 5. Write sample text to a file using fs.writeFileSync().
 6. Read the created file using fs.readFileSync().
 7. Display the results in the terminal.
 8. Run the program using node app.js.
-

6. PROGRAM

app.js

```
const os = require("os");
const path = require("path");
const fs = require("fs");

// os module
console.log("Operating System:", os.platform());
console.log("CPU Cores:", os.cpus().length);
console.log("Home Directory:", os.homedir());

// path module
const filePath = path.join(__dirname, "sample.txt");
```

```
console.log("File Path:", filePath);

// fs module
const content = "This file was created using the Node.js fs module.";
fs.writeFileSync(filePath, content, "utf8");

const fileData = fs.readFileSync(filePath, "utf8");
console.log("File Content:", fileData);
```

7. CODE EXPLANATION

Code	Explanation
require('os')	Imports the built-in os module.
os.platform()	Returns the operating-system platform.
os.cpus().length	Returns the number of available CPU cores reported by Node.js.
os.homedir()	Returns the user's home directory.
require('path')	Imports the built-in path module.
path.join()	Creates a platform-independent file path.
fs.writeFileSync()	Writes text to a file synchronously.
fs.readFileSync()	Reads file contents synchronously.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Test	Expected Result	Status
TC-01	node app.js	OS information is displayed	Pass
TC-02	Path operation	A valid sample.txt path is displayed	Pass
TC-03	File handling	sample.txt is created and its contents are displayed	Pass

9. OUTPUT

Output 1: Program Output Screenshot

```
Operating System: win32
CPU Cores: 12
Home Directory: C:\Users\hansi
File Path: C:\Users\hansi\OneDrive\Desktop\git-programs\full-stack\01-observation-and-record\observation\Task 6\Q6-NodeJS-Modules\src\sample.txt
File Content: This file was created using the Node.js fs module.
```

10. RESULT

Thus, the Node.js os, path, and fs built-in modules were successfully demonstrated for operating-system information, path handling, and file operations.

How this would be submitted

Source Code

Task_6/

■■■ app.js

■■■ README.md